Assignment 3 DESIGN.pdf:

Description of Program:

This program implements Insertion Sort, Shell Sort, Heap sort, and recursive Quick sort based on the provided Python pseudocode. The interface for these sorts were given as the header files insert.h, shell.h, heap.h, and quick.h. The test harness creates an array of pseudorandom elements and tests each of the sorts. Program will gather statistics about each sort and its performance. The statistics will be the size of the array, the number of moves required, and the number of comparisons required.

Files to be included in directory “asgn2” + pseudocode/structure:

- insert.c
    - Implements insertion sort
    - Insertion Sort compares the current element with each of the preceding elements in descending order until its position is found.
    - Use for loop to go through the array
- insert.h
    - specifies the interface to insert.c
- heap.c
    - Implements heap sort
    - The first routine is taking the array to sort and building a heap from it. This means ordering the array elements such that they obey the constraints of a max or min heap. For our purposes, the constructed heap will be amax heap. This means that the largest element, the root of the heap, is the first element

of the array from which the heap is built. The second routine is needed as we sort the array. The gist of Heap sort is that the largest array elements are repeatedly removed from the top of the heap and placed at the end of the sorted array, if the array is to be sorted in increasing order. After removing the largest element from the heap, the heap needs to be fixed so that it once again obeys the constraints of a heap.

- heap.h
    - specifies the interface to heap.c
- quick.c
    - implements recursive Quicksort.
    - Quick sort is a divide-and-conquer algorithm. It partitions arrays into two sub-arrays by selecting an element from the array and designating it as a pivot. Elements in the array that are less than the pivot goto the left sub-array, and elements in the array that are greater than or equal to the pivot go to the right sub-array.
- quick.h
    - specifies the interface to quick.c
- set.h
    - implements and specifies the interface for the set ADT.
- stats.c
    - implements the statistics module.
- stats.h
    - specifies the interface to the statistics module.
- shell.c

    - implements Shell Sort.
    - Shell Sort is a variation of Insertion Sort and first sorts pairs of elements which are far apart from each other. The distance between these pairs of elements is called a gap. Each iteration of Shell Sort decreases the gap until a gap of 1 is used.
- shell.h
    - specifies the interface to shell.c
- sorting.c
    - Contains main()
    - Your test harness must support any combination of the following command-line options:
        - -a: Employs all sorting algorithms
        - -e: Enables Heap Sort
        - -i: Enables Insertion Sort
        - -s: Enables Shell Sort
        - -q: Enables Quick sort
        - -r seed: Set the random seed to seed. The default seed should be 13371453
        - -n size: Set the array size to size. The default size should be 100
        - -p elements: Print out the number of elements from the array. The default number of elements to print out should be 100. If the size of the array is less than the specified number of elements to print, print out the entire array and nothing more
        - -h: Prints out program usage.

- WRITEUP.pdf
    - This document must be a proper PDF.
    - Graphs displaying the difference between the values reported by implemented functions and that of the math library's.
    - Use a UNIX tool to produce graphs. gnuplot is recommended.
- Makefile
    - CC = clang must be specified.
    - CFLAGS = -Wall -Wextra -Werror -Wpedantic must be specified.
    - make must build the Mathlib-test executable, as should make all and make mathli
    - make clean must remove all files that are compiler generated.
    - make format should format all your source code, including the header files.
- README.md
    - In proper Markdown syntax.
    - Describe how to use your program and Makefile
    - Lists and explains any command-line options that program accepts.
    - Reports any false positives by scan-build
    - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf
    - This document must be a proper PDF.
    - Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.